

SJ

中华人民共和国电子行业标准

XX/T XXXXX—XXXX

超算云环境下共性算法库软件技术要求

Technical requirements for common algorithm library software in supercomputing cloud environment

(点击此处添加与国际标准一致性程度的标识)

(工作组讨论稿)

在提交反馈意见时，请将您知道的相关专利连同支持性文件一并附上。

XXXX – XX – XX 发布

XXXX – XX – XX 实施

中华人民共和国工业和信息化部 发布

目 次

前 言 II

1 范围 1

2 规范性引用文件 1

3 术语和定义 1

4 缩略语 1

前 言

本文件按照GB/T 1.1—2020《标准化工作导则 第1部分：标准化文件的结构和起草规则》的规定起草。

本文件由××××提出。

本文件由××××归口。

本文件起草单位：中国科学院计算技术研究所、中国人民解放军国防科技大学、无锡江南计算技术研究所、湖南大学、曙光信息产业股份有限公司、国家超级计算天津中心、中国科学院软件研究所、清华大学、西北大学

本文件主要起草人：贾海鹏、李胜国、李芳、江颖、罗辉章

超算云环境下共性算法库软件技术要求

1 范围

本文件规定了超算云环境下共性算法库软件技术要求，涵盖功能规范、接口规范、测试规范和代码规范。

本文件适用于云计算、高性能计算、智能计算等技术的融合应用场景，包括科学计算、大模型、等领域，并可作为产品研发和第三方评测的通用依据。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中，注日期的引用文件，仅该日期对应的版本适用于本文件；不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

3 术语和定义

下列术语和定义适用于本文件。

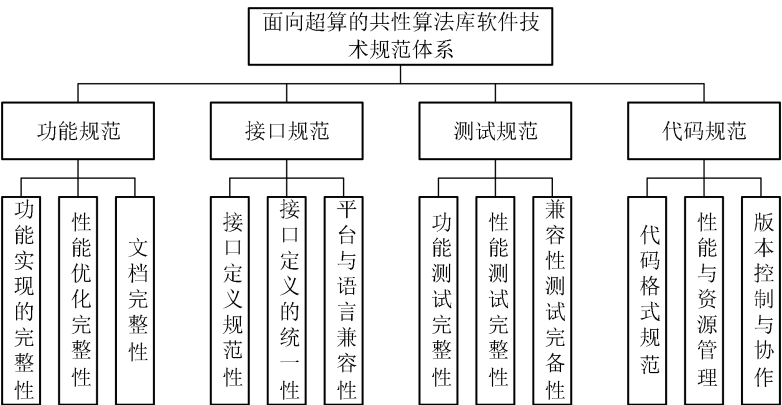
3.1

4 缩略语

下列缩略语适用于本文件。

5 软件规范体系

面向超算的共性算法库软件技术规范体系包括功能完整性、接口统一性、测试完备性、开发规范性。



6 功能规范

6.1 功能实现的完整性

共性算法库应包含如下算法子库或者子库子集：

- 1) 基本线性代数子程序（BLAS），包含向量-向量计算、矩阵-向量计算和矩阵-矩阵计算。具体如下：

	序号	算法名称	算法功能
BLAS LEVEL1	1	blas_?asum	用来计算一个向量的元素的模的总和。对于实数向量，计算向量中元素模（即绝对值）的总和；而对于复数向量，计算复数向量中元素的实部和虚部的模的总和。
	2	blas_?axpy	?axpy 例程执行一个向量-向量运算，用来计算一个向量 X 与一个标量 a 的乘积，并将结果加到另一个向量 Y 上，结果保存在 Y 向量中，即 $Y = a * X + Y$
	3	blas_?copy	?copy 例程执行一个向量-向量运算，用来将一个向量 X 中的元素复制到另一个向量 Y 中，即 $Y = X$
	4	blas_?dot	?dot 例程执行一个向量-向量运算，用来计算向量 X 和向量 Y 的点积。
	5	blas_?sdot	?sdot 例程执行一个向量-向量运算，用来计算单精度向量 X 和单精度向量 Y 的点积结果。在计算过程中，采用双精度进行计算。对于 sdsdot 函数，会在最终的点积结果中加上一个单精度标量值 sb。
	6	blas_?dotc_sub	?dotc_sub 例程执行一个向量-向量运算，用来计算复数向量 X 的共轭向量和复数向量 Y 的点积。
	7	blas_?dotu_sub	?dotu_sub 例程执行一个向量-向量运算，用来计算复数向量 X 和复数向量 Y 的点积。
	8	blas_?nrm2	?nrm2 例程用来计算一个向量的欧几里得范数，即 $Res = X $ ，其中，X 为要计算的向量；res 为计算结果，即函数返回值。
	9	blas_?rot	?rot 例程用来对平面内的点执行旋转操作，即 $X = c * X + s * Y$ $Y = c * Y - s * X$ 当 X 和 Y 为复数向量且 s 为复数类型标量时，则 $X = c * X + s * Y$ $Y = c * Y - conj(s) * X$ 其中，conj(s) 表示 s 的共轭。
	10	blas_?rotg	?rotg 例程根据给定点的笛卡尔坐标 (a, b)，计算出其 Givens 旋转相关的参数 c、s、r 和 z。 关于参数 z，当 $ a > b $ 时，其值为 s；否则，当 c 不为 0 时，其值为 $1/c$ ；否则其值为 1。
	11	blas_?rotm	?rotm 例程用来执行平面上的修正 Givens 旋转操作。 给定两个向量 X 和 Y，每个向量按如下方式被替换： $Trans([xi \ yi]) = H * Trans([xi \ yi])$ 即 $xi = h11 * xi + h12 * yi$ $yi = h21 * xi + h22 * yi$ 其中，xi 和 yi 是向量 X 和 Y 中的元素，Trans() 表示转置；H 是 2×2 的修正变换矩阵，h11, h12, h21 和 h22 是 H 的四个元素。
	12	blas_?rotmg	?rotmg 例程对于给定的输入向量的笛卡尔坐标 (x1, y1)，计算其一个修正 Givens 变换矩阵 H 的参数，使得该矩阵将变换后的结果的 y 分量清零。 变换形式为： $Trans([x1 \ 0]) = H * Trans([x1 * sqrt(d1) \ y1 *$

			$\sqrt{d_2}]$ 其中, x_1 和 y_1 表示输入的笛卡尔积, $\text{Trans}()$ 表示转置, $\sqrt{\quad}$ 表示开方; H 为 2×2 的矩阵变换矩阵;
	13	blas_?scal	?scal 例程用来将一个向量 X 的每个元素与一个标量 a 相乘, 即 $X = a * X$
	14	blas_?swap	?swap 例程用来将向量 X 中的元素逐一与向量 Y 中对应位置的元素做交换。
	15	blas_i?amax	i?amax 例程用来查找向量中绝对值最大的元素的索引。对于实数向量, 返回绝对值最大的元素的下标; 对于复数向量, 返回实部绝对值与虚部绝对值和最大的元素的下标。
	16	blas_i?amin	i?amin 例程用来查找向量中绝对值最小的元素的索引。对于实数向量, 返回绝对值最小的元素的下标; 对于复数向量, 返回实部绝对值与虚部绝对值和最小的元素的下标。
	17	blas_?cabs1	?cabs1 例程用来计算复数 z 的绝对值, 即 $\text{res} = \text{Re}(z) + \text{Im}(z) $ 其中, res 表示计算结果, 即返回值。
BLAS LEVEL2	18	blas_?gbmv	?gbmv 例程用来完成通用带状矩阵-向量乘法, 即 $y = \alpha * \text{op}(A) * x + \beta * y$ 其中, $\text{op}(A)$ 表示通用带状矩阵, x 和 y 表示向量; $\text{op}(A)$ 表示 $\text{op}(A) = A$ 、 $\text{op}(A) = A'$ 和 $\text{op}(A) = \text{conjg}(A')$ 中的一种; α 为标量;
	19	blas_?gemv	?gemv 例程用来完成通用矩阵-向量乘法, 即 $y = \alpha * \text{op}(A) * x + \beta * y$ 其中, $\text{op}(A)$ 表示通用矩阵, x 和 y 表示向量; $\text{op}(A)$ 表示 $\text{op}(A) = A$ 、 $\text{op}(A) = A'$ 和 $\text{op}(A) = \text{conjg}(A')$ 中的一种; α 为标量。
	20	blas_?ger	?ger 例程用来执行通用矩阵的 rank-1 更新, 即 $A = \alpha * x * y' + A$ 其中, A 表示 $M \times N$ 的通用矩阵, x 表示 M 维向量, y 表示 N 维向量, α 为标量。
	21	blas_?gerc	?gerc 例程用来执行通用矩阵的共轭 rank-1 更新, 即 $A = \alpha * x * \text{conjg}(y') + A$ 其中, A 表示 $M \times N$ 的通用矩阵, x 表示 M 维向量, y 表示 N 维向量, α 为标量。
	22	blas_?geru	?geru 例程用来执行通用矩阵的非共轭 rank-1 更新, 即 $A = \alpha * x * y' + A$ 其中, A 表示 $M \times N$ 的通用矩阵, x 表示 M 维向量, y 表示 N 维向量, α 为标量。
	23	blas_?hbmV	?hbmV 例程用来完成 Hermitian 带状矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 其中, α 和 β 是标量; A 表示 $N \times N$ 的具有 K 个上(或下)对角线的 Hermitian 带状矩阵; x 和 y 表示 N 维向量;
	24	blas_?hemv	?hemv 例程用来完成 Hermitian 矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 其中, α 和 β 是标量; A 表示 $N \times N$ 的 Hermitian 矩阵; x 和 y 表示 N 维向量;

25	blas_?her	?her 例程用来执行 Hermitian 矩阵的 rank-1 更新, 即 $A = \alpha * x * \text{conj}(x') + A$ 。其中, α 为实数标量, A 表示 $N \times N$ 的 Hermitian 矩阵, x 表示 N 维向量。
26	blas_?her2	?her2 例程用来执行 Hermitian 矩阵的 rank-2 更新, 即 $A = \alpha * x * \text{conj}(y') + \text{conj}(\alpha) * y * \text{conj}(x') + A$ 。其中, α 为标量, A 表示 $N \times N$ 的 Hermitian 矩阵, x 和 y 表示 N 维向量。
27	blas_?hpmv	?hpmv 例程用来完成 Hermitian 压缩矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 。其中, α 和 β 是标量; A 表示 $N \times N$ 的 Hermitian 矩阵, 以压缩形式存储; x 和 y 表示 N 维向量;
28	blas_?hpr	?hpr 例程用来执行 Hermitian 压缩矩阵的 rank-1 更新, 即 $A = \alpha * x * \text{conj}(x') + A$ 。其中, α 为实数标量; A 表示 $N \times N$ 的 Hermitian 矩阵, 以压缩形式存储; x 表示 N 维向量。
29	blas_?hpr2	?hpr2 例程用来执行 Hermitian 压缩矩阵的 rank-2 更新, 即 $A = \alpha * x * \text{conj}(y') + \text{conj}(\alpha) * y * \text{conj}(x') + A$ 。其中, α 为标量; A 表示 $N \times N$ 的 Hermitian 矩阵, 以压缩形式存储; x 和 y 表示 N 维向量。
30	blas_?sbmv	?sbmv 例程用来完成对称带状矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 。其中, α 和 β 为标量; x 和 y 为 N 维向量; A 为 $N \times N$ 的具有 K 个上(或下)对角线的对称带状矩阵。
31	blas_?spmv	?spmv 例程用来完成对称压缩矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 。其中, α 和 β 为标量; x 和 y 为 N 维向量; A 为 $N \times N$ 的对称矩阵, 以压缩形式存储。
32	blas_?spr	?spr 例程用来完成对称压缩矩阵的 rank-1 更新, 即 $A = \alpha * x * x' + A$ 。其中, α 为实数标量; A 表示 $N \times N$ 的对称矩阵, 以压缩形式存储; x 表示 N 维向量。
33	blas_?spr2	?spr2 例程用来完成对称压缩矩阵的 rank-2 更新, 即 $A = \alpha * x * y' + \alpha * y * x' + A$ 。其中, α 为标量; A 表示 $N \times N$ 的对称矩阵, 以压缩形式存储; x 和 y 表示 N 维向量。
34	blas_?symv	?symv 例程用来完成对称矩阵-向量乘法, 即 $y = \alpha * A * x + \beta * y$ 。其中, α 和 β 为标量; x 和 y 为 N 维向量; A 为 $N \times N$ 的对称矩阵。
35	blas_?syr	?syr 例程用来完成对称矩阵的 rank-1 更新, 即 $A = \alpha * x * x' + A$ 。其中, α 为实数标量; A 表示 $N \times N$ 的对称矩阵; x 表示 N 维向量。
36	blas_?syr2	?syr2 例程用来完成对称矩阵的 rank-2 更新, 即 $A = \alpha * x * y' + \alpha * y * x' + A$ 。其中, α 为标量; A 表示 $N \times N$ 的对称矩阵; x 和 y 表示 N 维向量。

	37	blas_?tbmv	?tbmv 例程用来完成三角带状矩阵-向量乘法，即 $x = \text{op}(A) * x$ 其中，op(A) 表示 N×N 的单位或单位的上或下三角带状矩阵；x 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。
	38	blas_?tbsv	?tbsv 例程用来求解系数为三角带状矩阵的线性方程组，即 $\text{op}(A) * x = b$ 其中，op(A) 表示 N×N 的单位或非单位的上或下三角带状矩阵；x 和 b 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。函数的解向量 b 会覆写到向量 x 中。该例程不进行奇异性或接近奇异性的测试。
	39	blas_?tpmv	?tpmv 例程用来完成三角压缩矩阵-向量乘法，即 $x = \text{op}(A) * x$ 其中，op(A) 表示 N×N 的单位或非单位的上或下三角矩阵，以压缩形式存储；x 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。
	40	blas_?tpsv	?tpsv 例程用来求解系数为三角压缩矩阵的线性方程组，即 $\text{op}(A) * x = b$ 。 其中，op(A) 表示 N×N 的单位或非单位的上或下三角矩阵，以压缩形式存储；x 和 b 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。函数的解向量 b 会覆写到向量 x 中。该例程不进行奇异性或接近奇异性的测试。
	41	blas_?trmv	?trmv 例程用来完成三角矩阵-向量乘法，即 $x = \text{op}(A) * x$ 其中，op(A) 表示 N×N 的单位或非单位的上或下三角矩阵；x 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。
	42	blas_?trsv	?trsv 例程用来求解系数为三角矩阵的线性方程组，即 $\text{op}(A) * x = b$ 其中，op(A) 表示 N×N 的单位或非单位的上或下三角矩阵；x 和 b 表示 N 维向量；op(A) 表示 op(A) = A、op(A) = A' 和 op(A) = conjg(A') 中的一种。函数的解向量 b 会覆写到向量 x 中。该例程不进行奇异性或接近奇异性的测试。
BLAS LEVEL3	43	blas_?gemm	?gemm 例程用来计算通用矩阵的标量-矩阵的矩阵-矩阵乘法，并将结果加到一个标量-矩阵乘积上，即 $C = \alpha * \text{op}(A) * \text{op}(B) + \beta * C$ 其中，op(X) 是 op(X) = X、op(X) = X_T (转置) 和 op(X) = X_H (共轭转置) 中的一种；alpha 和 beta 是标量；A、B 和 C 代表矩阵。
	44	blas_?hemm	?hemm 例程用来计算标量-矩阵-矩阵乘法，并将结果加到一个标量-矩阵乘积上。其中一个矩阵为 Hermitian 矩阵，即 $C = \alpha * A * B + \beta * C$

		或者 $C = \alpha * B * A + \beta * C$ 其中, α 和 β 是标量; A 是一个 Hermitian 矩阵; B 和 C 是一个 $M \times N$ 的矩阵。
45	blas_?herk	?herk 例程用来完成 Hermitian 矩阵的 rank-k 更新, 即 (示例一): $C = \alpha * A * A_H + \beta * C$ 或者 (示例二): $C = \alpha * A_H * A + \beta * C$ 其中, α 和 β 为实数标量; A 是一个 $N \times K$ 的矩阵 (在第一种示例中) 或者 $K \times N$ 的矩阵 (在第二种示例中), A_H 为其共轭转置矩阵; C 是一个 $N \times N$ 的 Hermitian 矩阵。
46	blas_?her2k	?her2k 例程用来完成 Hermitian 矩阵的 rank-2k 更新, 即 (示例一): $C = \alpha * A * B_H + \text{conjg}(\alpha) * B * A_H + \beta * C$ 或者 (示例二): $C = \alpha * A_H * B + \text{conjg}(\alpha) * B_H * A + \beta * C$ 其中, α 和 β 为标量且 β 为实数; A 和 B 是 $N \times K$ 的矩阵 (在第一种示例中) 或者 $K \times N$ 的矩阵 (在第二种示例中), 其共轭转置矩阵为 A_H 和 B_H ; C 是一个 $N \times N$ 的 Hermitian 矩阵。
47	blas_?symm	?symm 例程用来计算标量-矩阵-矩阵乘法, 并将结果加到一个标量-矩阵乘积上。其中一个矩阵为对称矩阵, 即 $C = \alpha * A * B + \beta * C$ 或者 $C = \alpha * B * A + \beta * C$ 其中, α 和 β 是标量; A 是一个对称矩阵; B 和 C 是一个 $M \times N$ 的矩阵。
48	blas_?syrk	?syrk 例程用来完成对称矩阵的 rank-k 更新, 即 (示例一): $C = \alpha * A * A' + \beta * C$ 或者 (示例二): $C = \alpha * A' * A + \beta * C$ 其中, α 和 β 为标量; A 是一个 $N \times K$ 的矩阵 (在第一种示例中) 或者 $K \times N$ 的矩阵 (在第二种示例中); C 是一个 $N \times N$ 的对称矩阵。
49	blas_?syr2k	?syr2k 例程用来完成对称矩阵的 rank-2k 更新, 即 (示例一): $C = \alpha * A * B' + \alpha * B * A' + \beta * C$ 或者 (示例二): $C = \alpha * A' * B + \alpha * B' * A + \beta * C$ 其中, α 和 β 为标量; A 和 B 是 $N \times K$ 的矩阵 (在第一种示例中) 或者 $K \times N$ 的矩阵 (在第二种示例中); C 是一个 $N \times N$ 的对称矩阵。
50	blas_?trmm	?trmm 例程用来计算标量-矩阵-矩阵乘法, 其中一

			个矩阵为三角矩阵，即 $C = \alpha * op(A) * B$ 或者 $C = \alpha * B * op(A)$ 其中，α 是标量；B 是一个 $M \times N$ 的矩阵；A 是一个单位或非单位的上或下三角矩阵。$op(A)$ 是 $op(A) = A$、$op(A) = A_T$（转置）和 $op(A) = A_H$（共轭转置）中的一种。
	51	blas_?trsm	?trsm 例程用来求解一个三角矩阵方程，即 $op(A) * X = \alpha * B$ 或者 $X * op(A) = \alpha * B$ 其中，α 是一个标量；X 和 B 是 $M \times N$ 的矩阵；A 是一个单位或单位的上或下三角矩阵；$op(A)$ 是 $op(A) = A$、$op(A) = A'$ 和 $op(A) = conjg(A')$ 中的一种。函数的解矩阵 X 会覆写到矩阵 B 中。
	52	blas_?trmm_oop	?trmm_oop 例程用来计算标量-矩阵-矩阵乘法，并将结果加到一个标量-矩阵乘积上。其中一个矩阵为三角矩阵，另一个矩阵为通用矩阵，即 $C = \alpha * op(A) * B + \beta * C$ 或者 $C = \alpha * B * op(A) + \beta * C$ 其中，α 和 β 是标量；A 是一个单位或非单位的上或下三角矩阵；B 和 C 是一个 $M \times N$ 的矩阵；$op(A)$ 是 $op(A) = A$、$op(A) = A'$ 和 $op(A) = conjg(A')$ 中的一种。
	53	blas_?trsm_oop	?trsm_oop 例程用来求解一个三角矩阵方程，并将结果加到另一个缩放矩阵中去，即 $op(A) * X = \alpha * B$ $C = X + \beta * C$ 或者 $X * op(A) = \alpha * B$ $C = X + \beta * C$ 其中，α 和 β 是标量；A 是一个单位或非单位的上或下三角矩阵；B 和 C 是一个 $M \times N$ 的矩阵；$op(A)$ 是 $op(A) = A$、$op(A) = A'$ 和 $op(A) = conjg(A')$ 中的一种。

2) 稠密线性代数并行算法库，包含线性方程组求解、最小二乘求解、特征值与奇异值求解以及西尔维斯特（Sylvester）方程求解。

	序号	算法名称	算法功能
Matrix Factorization	1	lapack_?getrf	?getrf 例程用来计算通用矩阵的 LU 分解，即 $A = P * L * U$ 其中，P 是置换矩阵；A 是 $M \times N$ 的通用矩阵；L 是单位对角元素的下三角矩阵（如果 $M > N$，则为下梯形）；U 是上三角矩阵（如果 $M < N$，则为上梯形）。

			该例程使用部分选主元法进行行交换。
2	lapack_?getrfnp		?getrfnp 例程用来计算通用矩阵的 LU 分解，但不进行选主元，即 $A = L * U$ 其中，A 是 M×N 的通用矩阵；L 是单位对角元素的下三角矩阵（如果 M>N，则为下梯形）；U 是上三角矩阵（如果 M<N，则为上梯形）。
3	lapack_?getrfnpi		?getrfnpi 例程用来计算通用矩阵的 LU 分解，但不进行选主元，且支持不完全分解。其分解形式为： $A = L * U$ 其中，A 是 M×N 的通用矩阵；L 是单位对角元素的下三角矩阵（如果 M>N，则为下梯形）；U 是上三角矩阵（如果 M<N，则为上梯形）。如果要进行不完全分解，其分解形式为： $A = L * U + A^{\sim}$ 其中，L 是单位对角元素的下梯形矩阵；U 是上梯形矩阵； $A^{\sim}$ 是矩阵 A 中未分解的部分。
4	lapack_?getrf2		?getrf2 例程用来计算通用矩阵的 LU 分解，并使用部分选主元进行行交换，即 $A = P * L * U$ 其中，P 是置换矩阵；A 是 M×N 的通用矩阵；L 是单位对角元素的下三角矩阵（如果 M>N，则为下梯形）；U 是上三角矩阵（如果 M<N，则为上梯形）。 该例程是算法的递归版本。
5	lapack_?gbtrf		?gbtrf 例程用来计算通用带状矩阵的 LU 分解，并使用部分选主元进行行交换（这会在 U 中产生额外的 k1 个上对角线），即 $A = P * L * U$ 其中，P 是置换矩阵；A 是 M×N 的通用带状矩阵，包含 k1 个非零下对角线和 ku 个非零上对角线；L 是单位对角元素的下三角矩阵，每列最多包含 k1 个非零元素；U 是有 k1+ku 个上对角线的上三角带状矩阵。
6	lapack_?gttrf		?gttrf 例程用来计算三对角矩阵的 LU 分解，并使用部分选主元和行交换，即 $A = L * U$ 其中，A 是三对角矩阵；L 是置换矩阵和单位对角元素的下三角矩阵的乘积；U 是上三角矩阵，只在主对角线和两个上对角线上有非零元素。
7	lapack_?dttrfb		?dttrfb 例程用来计算一个对角占优的三对角矩阵的 LU 分解，使用不进行选主元的BABE算法，即 $A = L1 * U * L2$ 其中，L1 和 L2 是单位下三角矩阵，分别有 k 和 k-n-1 个下三角元素；U 是一个双对角矩阵，只在主对角线和第一个对角线上有非零元素。
8	lapack_?potrf		?potrf 例程用来计算对称（或 Hermitian）正定矩阵的 Cholesky 分解。 对于实数矩阵，采用上三角形式， $A = U_T * U$ 采用下三角形式， $A = L * L_T$

		对于复数矩阵，采用上三角形式， $A = U_H * U$ 采用下三角形式， $A = L * L_H$ 其中，L 为下三角矩阵，L_T 是 L 的转置矩阵， L_H 是 L 的共轭转置矩阵；U 是上三角矩阵， U_T 是 U 的转置矩阵，U_H 是 U 的共轭转置矩阵。
9	lapack_?potrf2	?potrf2 例程使用一个递归的算法用来计算矩阵的 Cholesky 分解。 对于实数矩阵，采用上三角形式， $A = U_T * U$ 采用下三角形式， $A = L * L_T$ 对于复数矩阵，采用上三角形式， $A = U_H * U$ 采用下三角形式， $A = L * L_H$ 其中，L 为下三角矩阵，L_T 是 L 的转置矩阵， L_H 是 L 的共轭转置矩阵；U 是上三角矩阵， U_T 是 U 的转置矩阵，U_H 是 U 的共轭转置矩阵。 该例程是算法的递归版本。它将要分解的矩阵 A 分解为四个子矩阵：A11、A12、A21、A22。 其中，A 的维度为 N×N，A11 的维度为 N1×N1， A22 的维度为 N2×N2，且 N1=N/2，N2=N-N1。 子例程调用自身来分解 A11；然后更新并缩放 A12 和 A21；接着更新 A22，最后调用自身来分解 A22。
10	lapack_?pstrf	?pstrf 例程用来计算一个实对称（或复数 Hermitian）的半正定矩阵的 Cholesky 分解，使用完全选主元。 对于实数矩阵，采用上三角形式， $P_T * A * P = U_T * U$ 采用下三角形式， $P_T * A * P = L * L_T$ 对于复数矩阵，采用上三角形式， $P_T * A * P = U_H * U$ 采用下三角形式， $P_T * A * P = L * L_H$ 其中，P 是一个置换矩阵，存储为一个向量， P_T 是 P 的转置；L 为下三角矩阵，L_T 是 L 的转置矩阵， L_H 是 L 的共轭转置矩阵；U 是上三角矩阵， U_T 是 U 的转置矩阵，U_H 是 U 的共轭转置矩阵。 该算法不会检查矩阵 A 是否为半正定矩阵。 这个版本的算法调用了 Level 3 BLAS。
11	lapack_?pftrf	?pftrf 例程用来计算一个对称（或 Hermitian）的正定矩阵的 Cholesky 分解，使用 RFP (Rectangular Full Packed) 格式。 对于实数矩阵，采用上三角形式， $A = U_T * U$

		采用下三角形式， $A = L * L_T$ 对于复数矩阵，采用上三角形式， $A = U_H * U$采用下三角形式， $A = L * L_H$ 其中，A 采用RFP格式；L 为下三角矩阵，L_T 是 L 的转置矩阵，L_H 是 L 的共轭转置矩阵；U 是上三角矩阵，U_T 是 U 的转置矩阵，U_H 是 U 的共轭转置矩阵。这是算法的块（block）版本，这个版本的算法调用了 Level 3 BLAS。
12	lapack_?pptrf	?pptrf 例程用来计算一个对称（或 Hermitian）的正定矩阵的 Cholesky 分解，使用压缩存储格式。 对于实数矩阵，采用上三角形式， $A = U_T * U$ 采用下三角形式， $A = L * L_T$ 对于复数矩阵，采用上三角形式， $A = U_H * U$ 采用下三角形式， $A = L * L_H$ 其中，L 为下三角矩阵；U 是上三角矩阵，U_T 和 L_T 是 U 和 L 的转置，U_H 和 L_H 是 U 和 L 的共轭转置。
13	lapack_?pbtrf	?pbtrf 例程用来计算一个对称（或 Hermitian）的正定带状矩阵的 Cholesky 分解。 对于实数矩阵，采用上三角形式， $A = U_T * U$ 采用下三角形式， $A = L * L_T$ 对于复数矩阵，采用上三角形式， $A = U_H * U$ 采用下三角形式， $A = L * L_H$ 其中，L 为下三角矩阵；U 是上三角矩阵，U_T 和 L_T 是 U 和 L 的转置，U_H 和 L_H 是 U 和 L 的共轭转置。
14	lapack_?pttrf	?pttrf 例程用来计算一个对称（或 Hermitian）的正定三对角矩阵的分解。 对于实数矩阵， $A = L * D * L_T$ 对于复数矩阵， $A = L * D * L_H$ 其中，D 是对角矩阵，L 为单位下双对角矩阵，L_T 是 L 的转置，L_H 是 L 的共轭转置； 该分解也具有以下的形式： 对于实数矩阵， $A = U_T * D * U$ 对于复数矩阵 $A = U_H * D * U$

		其中，U 是单位双对角矩阵，U_T 是 U 的转置，U_H 是 U 的共轭转置。
15	lapack_?sytrf	?sytrf 例程计算一个实数/复数对称矩阵的 Bunch-Kaufman 分解，使用 Bunch-Kaufman 对角选主元，即 对于上三角形式 $A = U * D * U_T$ 对于下三角形式 $A = L * D * L_T$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵）U_T 和 L_T 是 U 和 L 的转置；D 是一个对称的块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
16	lapack_?sytrf_aa	?sytrf_aa 使用 Aasen 算法对一个对称矩阵进行分解，分解形式为： $A = U * T * U_T$ 或者 $A = L * T * L_T$ 其中，U（或者 L）是置换矩阵和单位上（下）三角矩阵的乘积，L_T 和 U_T 是 L 和 U 的转置；T 是一个复数对称三对角矩阵。 这是算法的块（block）版本，这个版本的算法调用了 Level 3 BLAS。
17	lapack_?sytrf_rook	?sytrf_rook 例程计算一个实数/复数对称矩阵的有界 Bunch-Kaufman 分解，有界 Bunch-Kaufman（“rook”）对角主元选取方法，即 对于上三角形式 $A = U * D * U_T$ 对于下三角形式 $A = L * D * L_T$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵），U_T 和 L_T 是 U 和 L 的转置；D 是一个对称的块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
18	lapack_?sytrf_rk	?sytrf_rk 例程使用有界 Bunch-Kaufman（rook）对角主元选取方法，对一个实数或复数对称矩阵 A 进行分解。其分解形式为： 若采用上三角形式 $A = P * U * D * (U_T) * (P_T)$ 若采用下三角形式 $A = P * L * D * (L_T) * (P_T)$ 其中 U（或 L）是单位对角的上（下）三角矩阵，U_T 和 L_T 是 U 和 L 的转置；P 是置换矩阵，P_T 是 P 的转置；D 是对称的块对角矩阵，包含 1×1 和 2×2 的对角块。 这是算法的块（block）版本，这个版本的算法调用了 Level 3 BLAS。

19	lapack_?hetrf	?hetrf 例程 计算一个复数 Hermitian 矩阵的 Bunch-Kaufman 分解，使用 Bunch-Kaufman 对角选主元，即 对于上三角形式 $A = U * D * U_H$ 对于下三角形式 $A = L * D * L_H$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵）U_H 和 L_H 是 U 和 L 的共轭转置；D 是一个 Hermitian 块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
20	lapack_?hetrf_aa	?hetrf_aa 例程 使用 Aasen 算法 对一个复数 Hermitian 矩阵进行分解。分解形式为： $A = U * T * U_H$ 或者 $A = L * T * L_H$ 其中，U（或者 L）是置换矩阵和单位上（下）三角矩阵的乘积，L_H 和 U_H 是 L 和 U 的共轭转置；T 是一个 Hermitian 三对角矩阵。 这是算法的块（block）版本，这个版本的算法调用了 Level 3 BLAS。
21	lapack_?hetrf_aa_work	?hetrf_aa_work 例程是 ?hetrf_aa 例程显示提供工作空间数组的版本。
22	lapack_?hetrf_rook	?hetrf_rook 例程 计算一个复数 Hermitian 矩阵的有界 Bunch-Kaufman 分解，使用有界 Bunch-Kaufman（“rook”）对角主元选取方法，即 对于上三角形式 $A = U * D * U_H$ 对于下三角形式 $A = L * D * L_H$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵），U_H 和 L_H 是 U 和 L 的共轭转置；D 是一个 Hermitian 块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
23	lapack_?hetrf_rk	?hetrf_rk 例程 使用有界 Bunch-Kaufman（rook）对角主元选取方法，对一个复数 Hermitian 矩阵 A 进行分解。其分解形式为： 若采用上三角形式 $A = P * U * D * (U_H) * (P_T)$ 若采用下三角形式 $A = P * L * D * (L_H) * (P_T)$ 其中 U（或 L）是单位对角的上（下）三角矩阵，U_H 和 L_H 是 U 和 L 的共轭转置；P 是置换矩阵，P_T 是 P 的转置；D 是 Hermitian 块对角矩阵，包含

			1×1和2×2的对角块。 这是算法的块（block）版本，这个版本的算法调用了 Level 3 BLAS。
	24	lapack_?sptrf	?sptrf 例程计算一个对称矩阵的 Bunch-Kaufman 分解，以压缩形式存储，即 对于上三角形式 $A = U * D * U_T$ 对于下三角形式 $A = L * D * L_T$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵）U_T 和 L_T 是 U 和 L 的转置；D 是一个对称的块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
	25	lapack_?hptrf	?hptrf 例程计算一个复数 Hermitian 矩阵的 Bunch-Kaufman 分解，以压缩形式存储，使用 Bunch-Kaufman 对角选主元，即 对于上三角形式 $A = U * D * U_H$ 对于下三角形式 $A = L * D * L_H$ 其中，A 是输入矩阵，U 和 L 是置换矩阵和单位三角矩阵的乘积（对于 U 是上三角矩阵，对于 L 是下三角矩阵）U_H 和 L_H 是 U 和 H 的共轭转置；D 是一个 Hermitian 块对角矩阵，包含 1×1 和 2×2 的对角块。U 和 L 包含与 D 中 2×2 对角块对应的 2×2 单位对角块。
	26	lapack_?spffrt2	?spffrt2 例程用于对对称矩阵 A 进行部分分解，即 $A = L * D * L_T$ 其中 L 为下三角矩阵，L_T 为 L 的转置；D 为对角矩阵；矩阵 A 以压缩（packed）格式存储，仅包含下三角部分。 该函数不进行主元置换（pivoting），不检查零对角元素。使用者需保证矩阵 A 可分解，否则结果可能错误，且无报错信息。
	27	lapack_?spffrtx	?spffrtx 例程与 ?spffrt2 例程功能基本一致，同样用于对对称矩阵 A 进行部分分解，即 $A = L * D * L_T$ 矩阵采用 packed 格式存储。L 为下三角矩阵，D 为对角矩阵。不同的是，spffrtx 可能包含一些性能优化或扩展特性，适用于更高效的场景，接口和使用方式完全一致。
Solving Systems of Linear Equations	28	lapack_?getrs	?getrs 例程用于求解具有能够进行 LU 分解的方阵系数矩阵的线性方程组，且该方程组包含多个右端项，即求解以下线性方程组的解 X： $\text{op}(A) * X = B$ 其中，op(A) 表示 op(A) = A（非转置）、op(A) = A_T（转置）和 op(A) = A_H（共轭转置）（仅对于复数）中的一种。

			在调用该例程之前，必须先调用?getrf 来计算 A 的 LU 分解。
29	lapack_?gbtrs		?gbtrs 例程用于求解具有能够进行 LU 分解的带状系数矩阵的线性方程组，且该方程组包含多个右端项，即求解以下线性方程组的解 X： $\text{op}(A) * X = B$ 其中，A 是一个具有 LU 分解的通用带状矩阵，阶数为 N，且具有 k1 个非零下对角线和 ku 个非零上对角线；op(A) 表示 $\text{op}(A) = A$（非转置）、$\text{op}(A) = A_T$（转置）和 $\text{op}(A) = A_H$（共轭转置）（仅对于复数）中的一种。 在调用该例程之前，必须先调用?gbtrf 来计算 A 的 LU 分解。
30	lapack_?gttrs		?gttrs 例程用于求解具有能够进行 LU 分解的三对角系数矩阵的线性方程组，即求解以下线性方程组的解 X： $\text{op}(A) * X = B$ 其中，op(A) 表示 $\text{op}(A) = A$（非转置）、$\text{op}(A) = A_T$（转置）和 $\text{op}(A) = A_H$（共轭转置）（仅对于复数）中的一种。 在调用该例程之前，必须先调用?gttrf 来计算 A 的 LU 分解。
	lapack_?dttrs		?dttrs 例程用于求解具有能够进行 LU 分解的对角占优的三对角系数矩阵的线性方程组，即求解以下线性方程组的解 X： $\text{op}(A) * X = B$ 其中，op(A) 表示 $\text{op}(A) = A$（非转置）、$\text{op}(A) = A_T$（转置）和 $\text{op}(A) = A_H$（共轭转置）（仅对于复数）中的一种。 在调用该例程之前，必须先调用?dttrf 来计算 A 的 LU 分解。
31	lapack_?potrs		?potrs 例程用于求解具有能够进行 Cholesky 分解的对称（或 Hermitian）正定系数矩阵的线性方程组，即求解以下线性方程组的解 X： $A * X = B$ 其中，A 是对称正定矩阵（对于复数数据，A 为 Hermitian 正定矩阵），并给定 A 的 Cholesky 分解为： 对于实数数据，当为上三角形式时： $A = U_T * U$ 或者对于下三角形式时： $A = L * L_T$ 对于复数数据，当为上三角形式时： $A = U_H * U$ 或者对于下三角形式时： $A = L * L_H$ 其中，L 为一个下三角矩阵，L_T 是 L 的转置，L_H 是 L 的共轭转置；U 是一个上三角矩阵，U_T 是 U 的转置，U_H 是 U 的共轭转置。该系统通过

			矩阵 B 的列存储的多个右项端来求解。执行前必须先调用?potrf 得到矩阵 A 的 Cholesky 分解。
32	lapack_?pftrs		?pftrs 例程用于求解具有能够进行 Cholesky 分解的对称（或 Hermitian）正定系数矩阵的线性方程组（该矩阵使用RFP格式），即使用 A 的 Cholesky 分解求解以下线性方程组的解 X： $A * X = B$ 其中，A 是对称正定矩阵（对于复数数据，A 为 Hermitian 正定矩阵），使用 RFP 格式，并给定 A 的 Cholesky 分解为： 对于实数数据，当为上三角形式时： $A = U_T * U$ 或者对于下三角形式时： $A = L * L_T$ 对于复数数据，当为上三角形式时： $A = U_H * U$ 或者对于下三角形式时 $A = L * L_H$ 其中，L 为一个下三角矩阵，L_T 是 L 的转置，L_H 是 L 的共轭转置；U 是一个上三角矩阵，U_T 是 U 的转置，U_H 是 U 的共轭转置。 执行前必须先调用 ?pftrf 得到矩阵 A 的 Cholesky 分解。
33	lapack_?pptrs		?pptrs 例程用于求解具有能够进行 Cholesky 分解的对称（或 Hermitian）正定系数矩阵的线性方程组（该矩阵使用压缩存储），即求解以下线性方程组的解 X： $A * X = B$ 其中，A 是对称正定矩阵（对于复数数据，A 为 Hermitian 正定矩阵），使用压缩存储，并给定 A 的 Cholesky 分解为： 对于实数数据，当为上三角形式时： $A = U_T * U$ 或者对于下三角形式时： $A = L * L_T$ 对于复数数据，当为上三角形式时： $A = U_H * U$ 或者对于下三角形式时 $A = L * L_H$ 其中，L 为一个下三角矩阵，L_T 是 L 的转置，L_H 是 L 的共轭转置；U 是一个上三角矩阵，U_T 是 U 的转置，U_H 是 U 的共轭转置。该系统通过矩阵 B 的列存储的多个右项端来求解。 执行前必须先调用 ?pptrf 得到矩阵 A 的 Cholesky 分解。
34	lapack_?pbtrs		?pbtrs 例程用于求解具有能够进行 Cholesky 分解的对称（或 Hermitian）正定带状系数矩阵的线性方程组，即求解以下线性方程组的解 X： $A * X = B$

		其中，A 是对称正定矩阵（对于复数数据，A 为 Hermitian 正定矩阵），给定 A 的 Cholesky 分解为： 对于实数数据，当为上三角形式时： $A = U_T * U$ 或者对于下三角形式时： $A = L * L_T$ 对于复数数据，当为上三角形式时： $A = U_H * U$ 或者对于下三角形式时： $A = L * L_H$ 其中，L 为一个下三角矩阵，L_T 是 L 的转置，L_H 是 L 的共轭转置；U 是一个上三角矩阵，U_T 是 U 的转置，U_H 是 U 的共轭转置。该系统通过矩阵 B 的列存储的多个右项端来求解。 执行前必须先调用 ?pbtrf 得到矩阵 A 的 Cholesky 分解，并以带状存储形式存储。
35	lapack_?pttrs	?pttrs 例程使用 ?pttrf 例程计算得到的分解，来求解具有对称（或 Hermitian）正定三对角系数矩阵的线性方程组，即求解以下线性方程组的解： $A * X = B$ 其中 A 为对称（或 Hermitian）正定三对角矩阵。 在调用这个例程之前，必须先调用 ?pttrf 例程来对矩阵 A 进行分解，分解形式为： 对于实数数据，有： $A = L * D * L_T$ 或者 $A = U_T * D * U$ 对于复数数据，有： $A = L * D * L_H$ 或者 $A = U_H * D * U$
36	lapack_?sytrs	?sytrs 用于求解具有能够进行 UDU_T 或 LDL_T 分解的对称系数矩阵的线性方程组，即求解以下线性方程组的解： $A * X = B$ 其中矩阵 A 采用 Bunch-Kaufman 分解形式： 对于上三角形式，有： $A = U * D * U_T$ 对于下三角形式，有： $A = L * D * L_T$ 其中 L 是具有单位对角线的下三角矩阵，L_T 是 L 的转置；U 是具有单位对角线的上三角矩阵，U_T 是 U 的转置；D 是一个对称的块对角矩阵。 该系统通过矩阵 B 的列中存储的多个右端项来求解。 调用该例程，必须向该例程提供 ?sytrf 例程分解返回的 U（或 L）和数组 ipiv。
37	lapack_?sytrs_aa	?sytrs_aa 例程用于求解具有对称矩阵 A 的线性方

		程组 $A * X = B$，使用通过?sytrf_aa 计算的分解 $U * T * U_T$ 或 $L * T * L_T$。 其中，L_T 为 L 的转置；U_T 为 U 的转置。
38	lapack_?sytrs_rook	?sytrs_rook 例程求解具有能够进行 UDU 或 LDL 分解的对称系数矩阵的线性方程组，即求解以下线性方程组的解： $A * X = B$。 该例程通过调用?sytrf_rook 例程来计算A的分解： $A = U * D * U_T$ 或者 $A = L * D * L_T$ 其中，L_T 为 L 的转置；U_T 为 U 的转置。
39	lapack_?hetrs	?hetrs 用于求解具有能够进行 UDU_T 或 LDL_T 分解的 Hermitian 系数矩阵的线性方程组，即求解以下线性方程组的解： $A * X = B$ 给定 Hermitian 矩阵 A 的 Bunch-Kaufman 分解形式： 对于上三角形式，有： $A = U * D * U_H$ 对于下三角形式，有： $A = L * D * L_H$ 其中 L 是具有单位对角线的下三角矩阵，L_H 是 L 的共轭转置；U 是具有单位对角线的上三角矩阵，U_H 是 U 的共轭转置；D 是一个对称的块对角矩阵。 该系统通过矩阵 B 的列中存储的多个右端项来求解。 调用该例程，必须向该例程提供?hetrf 例程分解返回的 U（或 L）和数组 ipiv。
40	lapack_?hetrs_aa	?hetrs_aa 例程用于求解具有复数 Hermitian 矩阵 A 的线性方程组 $A * X = B$，使用通过?hetrf_aa 计算的分解 $U * T * U_H$ 或 $L * T * L_H$。 其中，L_H 为 L 的共轭转置；U_H 为 U 的共轭转置。
41	lapack_?hetrs_aa_work	?hetrs_aa_work 例程是?hetrs_aa 例程显式提供工作空间数组的版本。
42	lapack_?hetrs_rook	?hetrs_rook 例程求解具有能够进行 UDU 或 LDL 分解的 Hermitian 系数矩阵的线性方程组，即求解以下线性方程组的解： $A * X = B$。 该例程通过调用?hetrf_rook 例程来计算复数 Hermitian 矩阵 A 的分解： $A = U * D * U_H$ 或者 $A = L * D * L_H$ 其中，L_H 为 L 的共轭转置；U_H 为 U 的共轭转置。
43	lapack_?sytrs2	?sytrs2 用于求解具有能够进行 UDU 或 LDL 分解的对称系数矩阵的线性方程组，即使用对称矩阵 A 的分解求解以下线性方程组的解：

		$A * X = B$ 给出对称矩阵 A 的分解： 对于上三角形式，有： $A = U * D * U_T$ 对于下三角形式，有： $A = L * D * L_T$ 其中 L 是具有单位对角线的下三角矩阵，L_T是L的转置；U是具有单位对角线的上三角矩阵，U_T是U的转置；D 是一个对称的块对角矩阵。该分解由?sytrf 计算得到。
44	lapack_?hetrs2	?hetrs2 用于求解具有能够进行 UDU 或 LDL 分解的 Hermitian 系数矩阵的线性方程组，即使用复数 Hermitian 矩阵 A 的分解求解以下线性方程组的解： $A * X = B$ 给出 Hermitian 矩阵 A 的分解： 对于上三角形式，有： $A = U * D * U_H$ 对于下三角形式，有： $A = L * D * L_H$ 其中 L 是具有单位对角线的下三角矩阵，L_H是L的共轭转置；U是具有单位对角线的上三角矩阵，U_H是U的共轭转置；D 是一个对称的块对角矩阵。该分解由?hetrf 计算得到。
45	lapack_?sytrs_3	?sytrs_3 用一个实数或者复数矩阵来求解线性方程组，即通过使用?sytrf_rk例程计算实数或者复数对称矩阵 A 的分解求解以下线性方程组的解： $A * X = B$ 给出对称矩阵 A 的分解： 对于上三角形式，有： $A = P * U * D * U_T * P_T$ 对于下三角形式，有： $A = P * L * D * L_T * P_T$ 其中 L 是具有单位对角线的下三角矩阵，L_T是 L 的转置；U是具有单位对角线的上三角矩阵，U_T是 U 的转置；P 是置换矩阵，P_T是 P 的转置；D 是一个对称的块对角矩阵，具有1×1和2×2的对角块。 该算法使用了 Level 3 BLAS。
46	lapack_?hetrs_3	?hetrs_3 用一个复数 Hermitian 矩阵来求解线性方程组，即通过使用?hetrf_rk 例程计算复数 Hermitian 矩阵 A 的分解求解以下线性方程组的解： $A * X = B$ 给出对称矩阵 A 的分解： 对于上三角形式，有： $A = P * U * D * U_H * P_T$ 对于下三角形式，有： $A = P * L * D * L_H * P_T$

		其中 L 是具有单位对角线的下三角矩阵，L_H 是 L 的共轭转置；U 是具有单位对角线的上三角矩阵，U_H 是 U 的共轭转置；P 是置换矩阵，P_T 是 P 的转置；D 是一个对称的块对角矩阵，具有 1×1 和 2×2 的对角块。 该算法使用了 Level 3 BLAS。
47	lapack_?sptrs	?sptrs 例程用于求解具有能够进行 UDU 或 LDL 分解的对称系数矩阵的线性方程组（该矩阵使用压缩存储），即求解以下线性方程组的解 X： $A * X = B$ 其中，A 是对称矩阵，使用压缩存储，并给定 A 的 Bunch-Kaufman 分解为： 当为上三角形式时： $A = U * D * U_T$ 或者对于下三角形式时： $A = L * D * L_T$ 其中，L 为一个压缩存储的单位下三角矩阵，L_T 是 L 的转置；U 是一个压缩存储的单位上三角矩阵，U_T 是 U 的转置。D 是一个对称的块对角矩阵。该系统通过矩阵 B 的列存储的多个右项端来求解。调用该例程，必须向该例程提供?sptrf 例程分解返回的 U（或 L）和数组 $ipiv$。
48	lapack_?hptrs	?hptrs 例程用于求解具有能够进行 UDU 或 LDL 分解的 Hermitian 系数矩阵的线性方程组（该矩阵使用压缩存储），即求解以下线性方程组的解 X： $A * X = B$ 其中，A 是 Hermitian 矩阵，并给定 A 的 Bunch-Kaufman 分解为： 当为上三角形式时： $A = U * D * U_H$ 或者对于下三角形式时： $A = L * D * L_H$ 其中，L 为一个压缩存储的单位下三角矩阵，L_H 是 L 的共轭转置；U 是一个压缩存储的单位上三角矩阵，U_H 是 U 的共轭转置。D 是一个对称的块对角矩阵。该系统通过矩阵 B 的列存储的多个右项端来求解。调用该例程，必须向该例程提供?hptrf 例程分解返回的 U（或 L）和数组 $ipiv$。
49	lapack_?trtrs	?trtrs 例程求解具有三角系数矩阵的线性方程组，且该方程组包含多个右端项，即求解以下线性方程组的解 X： $op(A) * X = B$ 其中，A 为三角矩阵；$op(A)$ 表示 $op(A) = A$（非转置）、$op(A) = A_T$（转置）和 $op(A) = A_H$（共轭转置）（仅对于复数）中的一种。
50	lapack_?tptrs	?tptrs 例程求解具有压缩存储的三角系数矩阵的线性方程组，且该方程组包含多个右端项，即求解以下线性方程组的解 X：

			$\text{op}(A) * X = B$ 其中, A 为压缩存储的三角矩阵; $\text{op}(A)$ 表示 $\text{op}(A) = A$ (非转置)、 $\text{op}(A) = A_T$ (转置) 和 $\text{op}(A) = A_H$ (共轭转置) (仅对于复数) 中的一种。
	51	lapack_?tbtrs	?tbtrs 例程求解具有带状三角系数矩阵的线性方程组, 且该方程组包含多个右端项, 即求解以下线性方程组的解 X : $\text{op}(A) * X = B$ 其中, A 为带状三角矩阵; $\text{op}(A)$ 表示 $\text{op}(A) = A$ (非转置)、 $\text{op}(A) = A_T$ (转置) 和 $\text{op}(A) = A_H$ (共轭转置) (仅对于复数) 中的一种。
Symmetric Eigenvalue Problems	52	lapack_?sytrd	?sytrd 例程通过正交相似变换将一个实对称矩阵 A 降为对称三对角矩阵 T , 即: $A = Q * T * Q_T$ 正交矩阵 Q 并不显示地形成, 而是表示为 $n-1$ 个初等反射矩阵的乘积。 Q_T 为 Q 的转置。
	53	lapack_?orgtr	?orgtr 例程用来生成由 ?sytrd 所确定的 $n \times n$ 的实正交矩阵 Q 。 该进程在调用 ?sytrd 进程后使用。
	54	lapack_?ormtr	?ormtr 例程将一个实矩阵与由 ?sytrd 例程确定的实正交矩阵 Q 相乘, 即: 当 Q 在实数矩阵 C 左侧时: $\text{op}(Q) * C$ 当 Q 在实数矩阵 C 右侧时: $C * \text{op}(Q)$ 其中, $\text{op}(Q)$ 为 $\text{op}(Q) = Q$ (非转置) 和 $\text{op}(Q) = Q_T$ (转置) 中的一种。 该进程在调用 ?sytrd 进程后使用。 计算结果被覆写到 C 中。
	55	lapack_?hetrd	?hetrd 例程通过单位相似变换将一个复数 Hermitian 矩阵 A 降为对称三对角矩阵 T , 即: $A = Q * T * Q_H$ 酉矩阵 Q 并不显示地形成, 而是表示为 $n-1$ 个初等反射矩阵的乘积。 Q_H 为 Q 的共轭转置。
	56	lapack_?ungtr	?ungtr 例程用来生成由 ?hetrd 所确定的 $n \times n$ 的复数酉矩阵 Q 。 该进程在调用 ?hetrd 进程后使用。
	57	lapack_?unmtr	?unmtr 例程将一复数矩阵与由 ?hetrd 例程确定的复数酉矩阵 Q 相乘, 即: 当 Q 在实数矩阵 C 左侧时: $\text{op}(Q) * C$ 当 Q 在实数矩阵 C 右侧时: $C * \text{op}(Q)$ 其中, $\text{op}(Q)$ 为 $\text{op}(Q) = Q$ (非转置) 和 $\text{op}(Q) = Q_H$ (共轭转置) 中的一种。 该进程在调用 ?hetrd 进程后使用。 计算结果被覆写到 C 中。

58	lapack_?sptdr	?sptdr 例程通过正交相似变换将一个压缩存储的实对称矩阵 A 降为对称三对角矩阵 T，即： $A = Q * T * Q_T$ 正交矩阵 Q 并不显示地形成，而是表示为 n-1 个初等反射矩阵的乘积。 Q_T 为 Q 的转置。
59	lapack_?opgtr	?opgtr 例程用来生成由?sptdr 所确定的 n×n 的实正交矩阵 Q。 该进程在调用?sptdr 进程后使用。
60	lapack_?opmtr	?opmtr 例程将一个实矩阵与由?sptdr 例程确定的实正交矩阵 Q 相乘，即： 当 Q 在实数矩阵 C 左侧时： $op(Q) * C$ 当 Q 在实数矩阵 C 右侧时： $C * op(Q)$ 其中，op(Q) 为 $op(Q) = Q$（非转置）和 $op(Q) = Q_T$（转置）中的一种。 该进程在调用?sptdr 进程后使用。 计算结果被覆写到 C 中。
61	lapack_?hptrd	?hptrd 例程通过单位相似变换将一个压缩存储的复数 Hermitian 矩阵 A 降为对称三对角矩阵 T，即： $A = Q * T * Q_H$ 酉矩阵 Q 并不显示地形成，而是表示为 n-1 个初等反射矩阵的乘积。 Q_H 为 Q 的共轭转置。
62	lapack_?upgtr	?upgtr 例程用来生成由?hptrd 所确定的 n×n 的复数酉矩阵 Q。 该进程在调用?hptrd 进程后使用。
63	lapack_?upmtr	?upmtr 例程将一复数矩阵与由?hptrd 例程确定的酉矩阵 Q 相乘，即： 当 Q 在实数矩阵 C 左侧时： $op(Q) * C$ 当 Q 在实数矩阵 C 右侧时： $C * op(Q)$ 其中，op(Q) 为 $op(Q) = Q$（非转置）和 $op(Q) = Q_H$（共轭转置）中的一种。 该进程在调用?hptrd 进程后使用。 计算结果被覆写到 C 中。
64	lapack_?sbtrd	?sbtrd 例程通过正交相似变换将一个实对称带状矩阵 A 降为对称三对角矩阵 T，即： $A = Q * T * Q_T$ 正交矩阵 Q 通过一系列 Givens 旋转的积来确定。 Q_T 为 Q 的转置。
65	lapack_?hbtrd	?hbtrd 例程通过单位相似变换将一个复数 Hermitian 带状矩阵 A 降为对称三对角矩阵 T，即： $A = Q * T * Q_H$ 酉矩阵 Q 通过一系列 Givens 旋转的积来确定。 Q_H 为 Q 的共轭转置。

	66	lapack_?sterf	?sterf 例程使用 QR 算法计算实对称矩阵T的所有特征值（该矩阵可以通过将对称矩阵或 Hermitian 矩阵化为三对角形式获得）。该例程使用不含平方根的 QR 算法变体。
	67	lapack_?steqr	?steqr 例程使用 QR 算法计算已化为三对角形式的对称或 Hermitian 矩阵的全部特征值和特征向量。在调用?steqr例程之前，必须通过调用以下例程先将矩阵化为三对角形并且生成矩阵Q。
	68	lapack_?stemr	?stemr 例程用来计算实对称三对角矩阵的部分（选定）特征值和特征向量。 正常执行?stemr 可能会产生 NaN 和无穷大，并且在不按照 IEEE 标准默认方式处理 NaN 和无穷大的环境中，可能会因为浮点异常而中止。
	69	lapack_?stedc	?stedc 例程使用分治法计算对称三对角矩阵的所有特征值和特征向量。如果使用 sytrd/hetrd 或 sptdr/hptrd 或 sbtrd/hbtrd 将矩阵化为三对角形，则也可以找到完整或带状实对称矩阵或 Hermitian 矩阵的特征向量。
	70	lapack_?stegr	?stegr 例程用来计算实对称三对角矩阵的部分（选定）特征值和特征向量。 该例程是围绕改进版?stemr 例程的兼容性封装。
	71	lapack_?pteqr	?pteqr 例程计算对称正定三对角矩阵的全部特征值以及（可选是否计算）全部特征向量。 在调用?pteqr例程之前，必须通过调用以下例程先将矩阵化为三对角形并且生成矩阵Q。
	72	lapack_?stebz	?stebz 例程使用二分法计算实对称三对角矩阵的选定特征值。
	73	lapack_?stein	?stein 例程通过迭代计算与指定特征值对应的实对称三对角矩阵T的特征向量。 该例程特别设计用于在调用?stebz 例程且 Order 为'B'计算出特征值之后使用，也可以在其他例程计算出特征值后使用。 如果 T 是通过将完整的对称或 Hermitian 矩阵 A 化为三对角形式得到的，那么可以调用 ?ormtr 或 ?opmtr（对于实数）以及?unmtr 或?upmtr（对于复数）来将 T 的特征向量转换为 A 的特征向量。
	74	lapack_?disna	?disna 例程用来计算对称/Hermitian 矩阵的特征向量，或一般 $m \times n$ 矩阵的左右奇异向量的倒数条件数（reciprocal condition numbers）。 该例程也可以用于计算广义对称特征问题的特征向量的误差界限。

3）稀疏基本线性代数子程序（Sparse BLAS），包含稀疏矩阵和向量的基本操作。

	序号	算法名称	算法功能
	1	sparse_?_create_csr	create_csr表示创建CSR格式的稀疏矩阵。
	2	sparse_?_create_csc	create_csc表示创建CSC格式的稀疏矩阵。

3	<code>sparse_?_create_coo</code>	<code>create_coo</code> 表示创建COO格式的稀疏矩阵。
4	<code>sparse_?_allocate_csr</code>	<code>allocate_csr</code> 表示只分配CSR格式稀疏矩阵所需的内存，没有初始化实际数据。
5	<code>sparse_?_allocate_csc</code>	<code>allocate_csc</code> 表示只分配CSC格式稀疏矩阵所需的内存，没有初始化实际数据。
6	<code>sparse_?_allocate_coo</code>	<code>allocate_coo</code> 表示只分配COO格式稀疏矩阵所需的内存，没有初始化实际数据。
7	<code>sparse_set_order</code>	对于CSR和CSC格式的矩阵，某些功能要求其每行/列元素按列下标从小到大的顺序存储，同理。由用户来初始化数据时，可能不满足有序要求，在初始化时可以使用辅助函数来标记是否有序（默认无序）。 <code>sparse_set_order</code> 会修改输入矩阵 <code>mat</code> 。将无序的CSR或CSC矩阵传入某些接口中时，可能会进行排序，从而修改非零元存储顺序。
8	<code>sparse_read_mtx</code>	稀疏矩阵初始化可从Matrix Market文件中读取矩阵。其中 <code>precision</code> 可取 <code>SPARSE_DOUBLE</code> 或 <code>SPARSE_SINGLE</code> , 表示矩阵的浮点精度。实数、复数类型由Matrix Market文件决定。
9	<code>sparse_destroy</code>	计算完成后用户可以销毁稀疏矩阵，释放 <code>malloc</code> 的矩阵内存空间。
10	<code>sparse_convert_coo</code>	将COO、CSR或CSC稀疏矩阵存储格式转换为COO格式。
11	<code>sparse_convert_csr</code>	将COO、CSR或CSC稀疏矩阵存储格式转换为CSR格式。
12	<code>sparse_convert_csc</code>	将COO、CSR或CSC稀疏矩阵存储格式转换为CSC格式。
13	<code>sparse_reorder_rows</code>	对稀疏矩阵行进行重排。主要用于三角分解与求解中，其排列会被保存起来，在求解时对应向量也会重排。排列向量有两种定义： 1. <code>perm</code> : 若将 <code>a</code> 重排成 <code>b</code> , 则 $b[i] = a[p[i]]$; 2. <code>iperm</code> : 若将 <code>a</code> 重排成 <code>b</code> , 则 $b[p[i]] = a[i]$ 。 矩阵重排支持CSR、CSC和COO。其中 <code>new_mat</code> 可以取 <code>SPARSE_IN_PLACE</code> , 表示操作后的结果写到 <code>old_mat</code> 里。
14	<code>sparse_reorder_columns</code>	对稀疏矩阵列进行重排。主要用于三角分解与求解中，其排列会被保存起来，在求解时对应向量也会重排。排列向量有两种定义： 1. <code>perm</code> : 若将 <code>a</code> 重排成 <code>b</code> , 则 $b[i] = a[p[i]]$; 2. <code>iperm</code> : 若将 <code>a</code> 重排成 <code>b</code> , 则 $b[p[i]] = a[i]$ 。 矩阵重排支持CSR、CSC和COO。其中

			new_mat 可以取 SPARSE_IN_PLACE, 表示操作后的结果写到 old_mat 里。
	15	sparse_reorder_row_col	对稀疏矩阵行列进行重排。
	16	sparse_?_perm_apply	对向量进行重排: 正向排列。
	17	sparse_?_iperm_apply	对向量进行重排: 逆向排列。
	18	sparse_?_elementwise_multiply	对向量进行逐元素放缩。
	19	sparse_?_trf	一阶段LU分解, 之后分别用于SpTRSV。要求A为CSC格式。 LU分解的低层次接口如下。所完成的功能是求 $\text{PrDrADcPc}^T = \text{LU}$ 后续的求解过程如下式, 其中除了三角求解以外, 还会用到sparse_?_iperm_apply、sparse_?_perm_apply 和 sparse_?_elementwise_multiply。 $x = A^{-1}b = \text{DcPc}^T U^{-1}L^{-1}\text{PrDrb}$ 输出参数中, lu表示因子矩阵 L 和 U, 其中 L 矩阵的对角线为1。实际上输出参数 lu 中存储的是 $L - I + U$, 且 U 的对角线是以倒数形式存储。riperm 和 ciperperm 是在分解过程中分别对进行的重排, 也就是排列阵 Pr 和 Pc; rscale 和 cscale 是分别对行列进行的放缩, 也就是对角阵 Dr 和 Dc, 注意对矩阵的放缩在重排之前。LU分解的并行版本是不可重现的: 同样的输入数据, 重复运行得到的多个结果之间不保证二进制相等, 但都在误差允许范围内。
Sparse BLAS LEVEL1	20	sparse_?_add	计算 $C = \alpha * A + \beta * B$, 要求A、B为CSR格式或CSC格式
Sparse BLAS LEVEL2	21	sparse_mv_inspect	SpMV 的计算分为两阶段, sparse_mv_inspect为第一阶段: 稀疏矩阵预处理, 根据稀疏矩阵的非零元个数判断是否进行矩阵格式转换。
	22	sparse_?_mv_execute	SpMV 的计算分为两阶段, sparse_?_mv_execute为第二阶段: 稀疏矩阵计算, 根据预处理后的矩阵进行SpMV计算。以实现: $y = \alpha * A * x + \beta * y$
	23	sparse_trsv_inspect	SpTRSV即对上三角或下三角矩阵 T 求解线性方程组: $T * y = \alpha * x$ 两阶段SpTRSV。要求A为CSR或CSC格式。options 必须是 SPARSE_LOWER_TRI 或 SPARSE_UPPER_TRI表示矩阵是下三角还是上三角。另外 options 可取 SPARSE_UNIT_DIAG 或 SPARSE_RECIP_DIAG, 通过按位或操作符“ ”

			与上下三角的标识同时传入。SPARSE_UNIT_DIAG 表示对角线全为 1,且在矩阵中没有被表示出来,也就是参数 a 只表示了矩阵的严格上/下三角部分;SPARSE_RECP_DIAG 表示对角线是否已经被取了倒数,这样会将除法替换为乘法。SpTRSV的并行版本是不可重现的:同样的输入数据,重复运行得到的多个结果之间不保证二进制相等,可能由于并行时序不同而导致求和顺序不同,但都在误差允许范围内。SpTRSV的低层次接口则可以暴露出这些中间数据。其中perm和iperm是在inspect的过程中对矩阵所做的重排,execute时输入的matrix为inspect输出的new_format。使用低层次接口时,注意要将输入和输出的稠密向量/矩阵做重排。
	24	sparse_?_trsv_execute	SpTRSV计算的第二阶段。
Sparse BLAS LEVEL3	25	sparse_mm_inspect	SpMM 的计算分为两阶段, sparse_mm_inspect为第一阶段:稀疏矩阵预处理,根据稀疏矩阵的非零元个数判断是否进行矩阵格式转换。
	26	sparse_?_mm_execute	SpMM 的计算分为两阶段, sparse_?_mm_execute为第二阶段:稀疏矩阵计算,根据预处理后的矩阵进行SpMM计算。以实现: $y = \alpha * A * x + \beta * y$
	27	?_gemm	SpGEMM表示稀疏矩阵与稀疏矩阵的乘积。计算: $C = \alpha * A * B + \beta * D$ 。要求A、B为CSR或者CSR格式,C的格式如表1所示。要求D的表示格式与C相同。若 $\beta = 0.0$,则输入参数 D 将被忽略,可以传入参数 SPARSE_MATRIX_NULL。

4) 稀疏直接解法器

	序号	算法名称	算法功能
稀疏直接法	1	dir_solver_?gssvx	求解AX=B。
	2	dir_solver_?gsequ	计算稀疏矩阵行列缩放因子,以对矩阵进行平衡来减少条件数。
	3	dir_solver_?laqgs	使用行列缩放因子对稀疏矩阵进行平衡。
	4	dir_solver_?CompRow_loc_to_CompCol_global	通过收集局部压缩行格式稀疏矩阵信息获得全局压缩列格式稀疏矩阵。
	5	dir_solver_?ldperm	稀疏矩阵行重排。
	6	dir_solver_symbfact	稀疏矩阵符号分解。

7	dir_solver_?distribute	将稀疏矩阵分布到二维进程网格。
8	dir_solver_?gstrf	并行LU分解。
9	dir_solver_?gsstrs	利用LU分解后的矩阵求解分布式稀疏线性系统。
10	dir_solver_?gsrfs	对稀疏线性系统的解进行改进。
11	dir_solver_?permute_dense_matrix	对右端项进行重排。
12	dir_solver_?static_schedule	对分解算法进行静态调度。
13	dir_solver_?redistribute_b_to_x	对右端项和解向量进行重新分布。

5) 稀疏迭代解法器

求解类别	求解方法	函数名称	函数功能
预条件迭代算法	结构化多重网格	结构PFMG	基于 PFMG算法, 求解线性方程组 $Ax=b$ 的函数
		结构SMG	基于平面平滑的半粗化多网格 (SMG) 求解器, 适用于三维问题线性方程组问题
		结构SparseMSG	该函数基于稀疏矩阵的多网格方法, 求解线性方程组 $Ax=b$
	半结构多重网格	半结构PFMG	该函数基于系统点松弛多网格方法, 求解线性方程组 $Ax=b$
		半结构Split	该函数基于分割求解方法 (Split Solver), 求解线性方程组 $Ax=b$
		半结构FAC	该函数基于全近似方案, 求解线性方程组 $Ax=b$
	代数多重网格	BoomerAMG	该函数使用 BoomerAMG (Algebraic Multigrid, 代数多重网格) 方法 来求解线性方程组
		BoomerAMGDD	该函数使用 BoomerAMG 方法结合双重分解技术求解稀疏线性方程组 $Ax=b$
	近似逆	FSAI	该函数使用 FSAI 方法求解稀疏线性方程组 $Ax=b$
		Parallel FSAI	该函数使用 ParaSails 方法求解稀疏线性方程组 $Ax=b$
		SPAI	该函数使用 SPAI 方法求解稀疏线性方程组 $Ax=b$
		Parallel SPAI	该函数通过并行化稀疏近似逆矩阵的计算稀疏线性方程组 $Ax=b$
	ILU	PILUT	该函数使用并行压缩稀疏行不完全LU分解 (PILUT) 方法求解稀疏线性方程组 $Ax=b$
		ILUT	该函数使用不完全LU分解方法求解稀疏线性方程组 $Ax=b$
基本迭代算法	高斯-赛德尔算法 (GaussSeidel)	GaussSeidel迭代函数	该函数通过迭代求解线性方程组的方法
	SOR迭代算法	SOR迭代函数	该函数通过迭代求解线性方程组的方法
	Richardson 迭代算法	Richardson迭代函数	该函数通过迭代的方式逐步逼近线性方程组的解

	Chebyshev迭代算法	Chebyshev迭代函数	一种多项式线性方程组求解迭代方法，特别适用于对称正定矩阵。通过 Chebyshev 多项式加速收敛
	Jacobi迭代算法	Struct Jacobi迭代函数	基于雅可比迭代方法，求解线性方程组 $Ax=b$
		并行的块Jacobi迭代函数	该函数使用 Euclid 预处理器求解稀疏线性方程组 $Ax=b$
子空间迭代算法	CG	PCG	求解对称正定线性方程组 $Ax=b$ 的共轭梯度算法的实现
		多右端共轭梯度法 (DCGMRHS)	求解多个右侧向量的对称正定线性方程组 $AX=B$ 的共轭梯度法的实现
		结构PCG	该函数使用共轭梯度法求解线性方程组 $Ax=b$
		半结构PCG	该函数基于预处理共轭梯度法，求解线性方程组 $Ax=b$
		(ParCSR) PCG	该函数使用共轭梯度方法求解稀疏线性方程组 $Ax=b$
		共轭梯度法求解正规方程函数 (CGNR)	使用共轭梯度法求解正规方，从而间接求解线性方程组 $Ax=b$
		Pipelined CG	求解线性方程组的解，通过减少同步操作来提高并行效率
		Pipelined Lanczos PCG	求解线性方程组的解，结合预处理技术和 Pipelined技术，提高并行效率
		信赖域共轭梯度求解函数	求解线性方程组的解，结合信赖域方法和共轭梯度法
		Pipelined Flexible PCG	求解线性方程组的解，通过减少同步操作来提高并行效率
	GMRES	GMRES	使用广义最小残差法 (GMRES) 求解稀疏线性方程组 $Ax=b$
		Flexible GMRES	求解一般线性方程组 $Ax=b$ 的灵活广义最小残差法的实现
		结构GMRES	该函数使用 GMRES 方法求解线性方程组 $Ax=b$
		结构Flexible GMRES	该函数通过最小化残差的范数来逐步逼近方程组的解
		结构化Local GMRES	该函数基于局部广义最小残差法求解线性方程组 $Ax=b$
		半结构GMRES	该函数基于广义最小残差法，求解线性方程组 $Ax=b$
		半结构Flexible GMRES	该函数基于灵活广义最小残差法，求解线性方程组 $Ax=b$
		半结构Local GMRES	该函数基于局部广义最小残差法，求解线性方程组 $Ax=b$ 的函数
		ParallelCSR GMRES	该函数使用 GMRES 方法求解稀疏线性方程组 $Ax=b$
		ParallelCSR Flexible GMRES	该函数使用Flexible GMRES 方法求解稀疏线性方程组 $Ax=b$
		共轭GMRES	使用共轭残差广义最小残差法 (COGMRES) 求解稀疏线性方程组 $Ax=b$

		Local GMRES	结合局部更新和全局更新，求解线性方程组的解
	BiCGStab	结构BiCGStab	该函数基于BiCGSTAB，求解线性方程组 $Ax=b$
		半结构化BiCGStab	该函使用BiCGSTAB进行迭代求解线性方程组 $Ax=b$
		ParallelCSR BiCGStab	该函数使用BiCGSTAB求解稀疏线性方程组 $Ax=b$
		BiCGStab	使用BiCGSTAB求解稀疏线性方程组 $Ax=b$
		Flexible BiCGStab	使用Flexible BiCGStab求解稀疏线性方程组 $Ax=b$
		Flexible BiCGStab(带重正交化)	使用Flexible BiCGStab求解稀疏线性方程组 $Ax=b$
	MINRES	MINRES 算法求解函数	通过最小化残差来逐步逼近稀疏线性方程组 $Ax=b$ 的解

6) 人工智能混合精度训练算法库

求解类别	类名称和功能	类成员类别	类成员名称	功能描述
Physic 库	Phy_Problem 类，定义待求解的pde问题的基类	类属性	collected_data	返回问题条件收集的数据
			input_pts	返回一个字典，将条件名称映射到其对应的输入点
			discretised_domains	返回一个字典，将域映射到其对应的采样点
			are_all_domains_discretised	检查所有域是否已离散化
			conditions	获取问题的输入变量
		方法	discretise_domain	获取问题的输出变量（抽象属性）
			add_points	向已采样的域添加新点

			collect_data	将问题条件的数据聚合到一个字典中
	Phy_Domain 类，定义待求解的pde问题 中输入点所在的几何区域的抽象基类	类属性	variables	返回域变量
			sample_modes	定义采样方法
		方法	sample	用于采样生成数据的过程
			is_inside	检查一个点是否在区域内
	Phy_SolverInterface 类，是定义科学智算求解器的抽象基类	类属性	problem	问题实例
			use_lt	在训练期间使用 LabelTensors 作为输入
			weighting	权重方案
		方法	forward	用于前向传播实现，以及输出训练后模型的预测值
			optimization_cycle	求解器的优化循环
			training_step	求解器训练步骤，计算优化循环并使用权重属性聚合损失
			validation_step	求解器验证步骤，计算优化循环并平均损失，未使用权重属性进行聚合
			test_step	求解器测试步骤，计算优化循环并平均损失，未使用权重属性进行聚合
			store_log	存储求解器的日志
			setup	在训练和测试过程开始时调用此方法，以在 Trainer compile 为 True 时编译模型
			get_batch_size	获取批次大小
			default_torch_optimizer	返回默认优化器设置，为 torch.optim.Adam
			default_torch_scheduler	返回默认调度器设置，为 torch.optim.lr_scheduler.Constant

				LR
Phy_PINN, 物理信息神经网络 (PINN) 科学智算求解器类	类属性	problem	要解决的问题实例	
		model	要使用的神经网络模型	
		optimizer	要使用的优化器。如果为 None，则使用 torch.optim.Adam 优化器，默认值为 None	
		scheduler	学习率调度器。如果为 None，则使用 torch.optim.lr_scheduler.Constant LR 调度器，默认值为 None	
		weighting	要使用的权重方案。如果为 None，则不使用权重方案，默认值为 None	
		loss	要最小化的损失函数。如果为 None，则使用 torch.nn.MSELoss 损失，默认值为 None	
	方法	loss_data	计算 PINN 求解器的数据损失，通过评估网络输出与真实解之间的损失。此方法不应被重写，除非有意为之。	
		loss_phys	根据提供的样本和方程计算物理损失	
		configure_optimizers	PINN 求解器的优化器配置	
	Phy_Trainer, 自定义科学智算求解器训练的基类	类属性	solver	获取求解器
方法		train	管理求解器的训练过程	
		test	管理求解器的测试过程	
Phy_LabelTensor 类, 用于定义待求解 pde 问题的采样数据	属性	tensor	返回 LabelTensor 对象的张量部分。	
		full_labels	返回张量的完整标签，即使对于未标记的维度。	
		stored_labels	返回存储在实例中的标签。	
		labels	返回实例最后一维的标签。	

		方法	<code>_init_labels_from_dict(labels)</code>	根据输入值存储内部标签表示。
			<code>_init_labels_from_list(labels)</code>	更新内部标签表示，通过将 <code>dof</code> 分配给最后一维。
			<code>extract(labels_to_extract)</code>	提取原始张量的子集。
			<code>static_cat(tensors, dim=0)</code>	沿指定维度连接张量列表。
			<code>static_stack(tensors)</code>	沿新维度堆叠张量列表。
			<code>requires_grad_(mode=True)</code>	重写 <code>requires_grad_()</code> 方法以处理新张量中的标签。
			<code>property dtype</code>	返回张量的数据类型。
			<code>to(*args, **kwargs)</code>	执行张量 <code>dtype</code> 和/或设备转换。
			<code>clone(*args, **kwargs)</code>	克隆 <code>LabelTensor</code> 。
			<code>append(tensor, mode='std')</code>	将给定张量附加到当前张量的最后一维。
			<code>static_vstack(tensors)</code>	垂直堆叠张量。
			<code>_update_single_label(index, dim)</code>	根据索引更新张量的标签。

			sort_labels(dim=None)	沿指定维度排序标签并应用。
			permute(*dims)	重新排列张量的维度及其相关标签。
			detach()	从计算图中分离张量并保留存储的标签。
			static summation(tensors)	计算 LabelTensor 实例的总和。
			reshape(*shape)	更新张量的形状和标签。
	LLM-LBO 类， 用于解决训练负载均衡问题	类属性	cluster_topo	集群拓扑结构，包括加速卡间互连特性
			perf_model	用于分析的性能模型
			parameters	模型参数量
		方法	dync_switch()	动态调整混合专家模型的放置位置
			balanced_expert()	针对热点专家，进行负载均衡的混合专家调度
	LLM-CCS 类， 用于解决集合通信优化问题	类属性	cluster_topo	集群拓扑结构，包括加速卡间互连特性
			perf_model	用于分析的性能模型
			parameters	模型参数量
		方法	topo_aware_all_reduce()	感知硬件网络拓扑的集合 all-reduce 通信操作，高效适配异构网络架构。
	LLM-DSS 类， 用于参数的高效分布式存储	类属性	cluster_topo	集群拓扑结构，包括加速卡间互连特性
			parameters	模型参数量
		方法	parameter_shard()	在模型训练优化器中，确定模型参数划分策略

	LLM-MAES 类， 用于高效内存感知执行	类属性	parameter_load ()	在计算时，负责聚合存储于其他设备的模型参数
			cluster_topo	集群拓扑结构，包括加速卡间互连特性
			parameters	模型参数量
		方法	parallel_strategy	模型并行训练策略
			memory_management ()	统一管理模型划分与放置策略，充分利用加速硬件内存
			placement_aware_schedule ()	根据放置策略调度计算的执行顺序，减少等待开销

7) 前后处理工具箱

求解类别	模块功能	函数名称	函数功能
网格生成	粗网格分区	PartitionMesh	使用METIS划分对输入体网格进行区域划分，生成每个单元所属子区域编号
		PartitionMetis	调用Metis进行区域分解，返回体网格所属子区域
	提取子网格	ExtractPartitionSurfaceMesh	获取面单元信息和子区域从属关系，提取并记录不同分区之间的共享面信息
		GetSurfPoints	从网格结构中提取表面点并填充到映射表中，用于后续构建表面元素
		ExtractSurfaceMesh	从完整网格中获取index为fid的面单元上点与子区域关系，用于构造多分区网格之间的边界信息
		ExtractFaceOutward	向外提取面，判断四面体外表面是否为外向（即是否为边界面），如果为外向面，返回 1，否则返回 0。
		PartFaceCreate	提取与MPI ID对应的子区域面网格，创建对应的接口面网格结构。
		FindSurfElem	返回顶点为elem的几何面index。
	细化网格	Refine	执行表面网格的递归细化处理，加密面网格。
		Refineforvol	对每个表面三角面和体单元进行中点插值，加密体网格。
	网格通信界面映射	InitBarycv	初始化重心坐标（Barycentric）结构
		SortInt	数组排序，整个数组元素将按升序

			排序。
		BarycMidPoint	计算两个重心坐标(Barycentric)的中点。
		computeadj	在并行网格划分中建立重心点与邻接子区域(进程)之间的映射关系表,并将其记录到barycvrtx2adjprocsmap中。
		com_barycoords	传递交界面节点,实现共享点全局ID分配;返回新全局点ID映射。
		com_baryVolumeElements	交换映射区体单元信息,更新体/点全局ID并记录插值单元。
数据压缩 可视化	数据压缩	zfp_stream_set_rate	设置压缩速率(每个标量的压缩比特数)
		zfp_stream_set_precision	设置压缩为定精度模式(每个标量的未压缩比特数)。
		vis_zfp_stream_set_accuracy	设置压缩为定容差模式(绝对误差容限)。
		zfp_stream_set_reversible	设置为无损(可逆)压缩模式。
		zfp_stream_set_params	以专家模式手动设置所有压缩参数。
		zfp_stream_set_bit_stream	将比特流与压缩流对象关联。
		zfp_stream_set_execution	设置压缩/解压的执行策略(串行、OpenMP、CUDA)。
		zfp_compress	对给定的未压缩数据域field进行压缩,结果写入stream关联的比特流。
		zfp_decompress	从stream关联的比特流中解压缩数据,写入到field指定的内存区域。
		zfp_encode_block_float_3 zfp_decode_block_float_3	对单个完整的3D 4x4x4浮点块进行压缩/解压缩。
		zfp_encode_block_strided_double_2 zfp_decode_block_strided_double_2	对非连续(带步长)内存布局的数据块进行压缩/解压缩。
		zfp_encode_partial_block_strided_float_3 zfp_decode_partial_block_strided_float_3	对部分填充的块(如边界块)进行压缩/解压缩。
数据压缩	数据可视	OpenDataFile	读取支持格式的数据文件(VTK、VTU、

可视化	化		PVD、CSV 等)
		GetTimeSteps	获取当前数据源的所有时间步
		Contour	根据标量字段生成等值面
		Slice	按平面切割数据
		Threshold	根据标量值范围提取数据
		Show	在渲染视图中显示数据对象
		Hide	隐藏数据对象
		Render	重新渲染当前视图
		SaveScreenshot	将当前视图保存为图片文件
		SaveData	将数据导出为文件（VTK、CSV 等）
		GetAnimationScene	获取动画场景对象，控制播放、时间范围等
		SaveAnimation	将动画保存为视频或图片序列

- 共性算法库应支持如下功能：
- 1) 支持多种内存布局：支持行主序和列主序两种不同的矩阵内存布局；
 - 2) 支持多精度：支持float、double、float complex、double complex四种精度；
 - 3) 支持异常处理：通过返回值或者返回错误代码处理异常情况；
 - 4) 支持资源管理：如果接口涉及资源分配（如内存），应提供相应的分配和释放接口；
 - 5) 支持用户调优：通过参数或配置选项允许用户优化性能（如设置线程数、缓存大小等）；
 - 6) 支持跨平台：确保接口在不同操作系统和硬件架构上行为一致；
 - 7) 支持多语言：提供统一的接口设计，便于为不同编程语言（如 C、C++、Python、Fortran）生成绑定；
 - 8) 支持版本管理：通过版本号管理算法库的变更，确保向后兼容。

6.2 性能优化完整性

共性算法库性能优化应面向底层硬件架构特征和应用特征，具备性能优化完整性，具体包含如下三个方面：

- 1) 硬件适配性优化
 - 支持国产超算的众核异构架构；
 - 支持针对国产超算众核加速器的性能优化，充分发挥众核加速器算力。
- 2) 多层次并行优化
 - 支持多级并行模型，包括MPI节点间并行、OpenMP/Pthread片内并行、SIMD核内并行、以及加速器并行，最大限度发挥超算的算力水平；
 - 支持针对硬件架构特征的多层次访存优化；
 - 支持混合精度优化；
 - 支持负载均衡优化，提高并行效率；

- 支持通信优化，通过异步通信、双缓冲等技术，用计算隐藏通信或者访存延迟。
- 3) 算法层面优化
- 支持算法复杂度优化，包括时间复杂度降低和空间复杂度降低等；
 - 支持面向应用特征的稀疏数据结构优化，包括压缩策略优化和分块稀疏编码优化等；
 - 支持数据规模自适应的调优方法，包括分块优化、多规模计算kernel优化等。

6.3 文档完整性

共性算法库应具备文档完整性，具体包含如下三个方面：

- 1) 用户使用手册：主要包含两部分内容：1) 共性算法库安装和调用方法；2) 接口描述，详细描述每个接口，包括接口定义、功能描述、参数说明、返回值说明、错误代码说明等；
- 2) 快速开发手册：提供共性算法库调用代码示例，帮助用户快速上手；提供性能优化指南，给出性能优化建议和使用场景说明；
- 3) 测试文档：详细说明测试环境和测试对象，提供功能测试、精度测试及性能测试的详尽数据等。

7 接口规范

7.1 接口定义的规范性

共性算法库接口定义应该统一命名规范，具体而言：

- 1) 功能单一性：每个接口应只完成一个明确的任务，避免功能冗余。
- 2) 命名一致性：接口命名应遵循统一的命名规则，应使用前缀表示功能类别，如 LibName_blas_ 表示“LibName”算法库下的BLAS 函数；
- 3) 可读性：函数名应清晰表达其功能，例如 LibName_blas_dgemm 表示双精度通用矩阵乘法；
- 4) 参数顺序：参数顺序保持一致，输入参数在前，输出参数在后；
- 5) 明确参数的数据类型（如 double、float、int 等），并避免隐式类型转换；
- 6) 使用枚举或常量定义标志参数，避免直接使用魔数（magic numbers）；
- 7) 统一错误码：定义一套统一的错误码，并在接口中返回；
- 8) 弃用策略：如果需要废弃某个接口，应提供明确的替代方案和迁移方案。

7.2 接口定义的统一性

共性算法库应定义统一用户接口，隐藏底层硬件的差异性，支持多种不同的硬件架构。（接口定义作为附件）

LibName_blas_?func, 包含四种情况: LibName_blas_sfunc, LibName_blas_dfunc, LibName_blas_cfunc, LibName_blas_zfunc;

T的定义:

- 1) 对于sfunc, T=float;
- 2) 对于dfunc, T=double;
- 3) 对于cfunc, T=LibNAME_Complex_Float;
- 4) 对于zfunc, T=LibNAME_Complex_Double.

- 1) 对于稠密基本线性代数算法库（BLAS），以gemm为例说明接口的统一定义（全写）

```
void LibName_blas_?gemm(
    LibNAME_Handle_t    Handle,
    LibNAME_ORDER Order,
    LibNAME_TRANSPOSE TransA,
    LibNAME_TRANSPOSE TransB,
    const LibNAME_INT M,
```



```

const LibNAME _INT N,
const LibNAME _INT K,
const T alpha,
const T *A,
const LibNAME _INT lda,
const T *B,
const LibNAME _INT ldb,
const T beta,
T *C,
const LibNAME _INT ldc
);

```

具体参数说明如下：

1. **Handle**, 进行BLAS计算的上下文
2. **Order**: 矩阵存储顺序, blasRowMajor (行主序) 或 blasColMajor (列主序)。
3. **TransA/TransB**: 矩阵转置选项, 可选 blasNoTrans (不转置)、blasTrans (转置) 或 blasConjTrans (共轭转置, 实数下同转置)。
4. **M/N/K**: 矩阵维度, 计算 $C = \alpha * \text{op}(A) * \text{op}(B) + \beta * C$, 其中:
 - $\text{op}(A)$ 是 $M \times K$ 矩阵 (若未转置) 或 $K \times M$ 矩阵 (若转置)。
 - $\text{op}(B)$ 是 $K \times N$ 矩阵 (若未转置) 或 $N \times K$ 矩阵 (若转置)。
 - C 是 $M \times N$ 矩阵。
5. **alpha/beta**: 标量系数, 参与乘法和累加运算。
6. **A/B/C**: 矩阵数据的指针, 按行主序或列主序存储。
7. **lda/ldb/ldc**: 矩阵的前导维度 (Leading Dimension), 即存储矩阵时相邻行 (行主序) 或列 (列主序) 之间的元素间隔。

7.3 平台与语言兼容性

共性算法库具备平台与语言兼容性, 具体而言:

1) 跨平台支持:

确保代码在不同操作系统 (如 Linux、Windows) 和硬件架构 (如 x86、ARM、SunWay) 上都能编译和运行。

2) 编译器兼容性:

使用标准语法, 避免依赖特定编译器的扩展功能。

3) 多语言绑定:

支持多语言绑定 (如 C/C++、Fortran), 确保接口设计一致。

8 测试规范

共性算法库的测试同时应具备功能测试完整性、性能测试完整性以及兼容性测试完整性。

8.1 测试前置条件

- 1) 应编写测试文档, 包含对所有功能和边界条件的测试;
- 2) 确定基准实现, 使用已知正确的参考实现进行结果对比, 确保算法实现的正确性;
- 3) 确定测试数据集, 采用业内认定的有效数据集, 确定测试结果的有效性;
- 4) 确定验证精度, 对于单精度浮点数, 要保证6~7位有效数字的正确性; 对于双精度浮点数, 要保证15~16位有效数字的正确性。

8.2 功能测试完整性

功能测试需对每个算法或者模块编写单元测试进行完整的功能测试，主要包含如下几个方面：

- 1) 随机测试, a) 随机输入测试, 使用随机生成的输入数据测试算法的鲁棒性。b) 模糊测试: 生成随机或异常输入, 测试库的稳定性和安全性;
- 2) 边界条件测试, a) 极小值测试, 测试输入为极小值或者空值的行为。b) 极大值测试: 测试输入为极大值时的行为。c) 特征值测试, 包含输入为特征值, 如NaN、Inf、-Inf时的行为;
- 3) 异常处理测试, a) 非法输入测试, 测试非法输入 (如空指针、负数维度、不匹配的矩阵维度) 时的行为。b) 错误码验证, 验证函数在异常情况下是否返回正确的错误码或状态。c) 资源耗尽测试, 测试在内存不足或其他资源耗尽时的行为;
- 4) 用户场景测试。a) 实际应用测试, 在真实应用场景中测试算法库精度。b) 压力测试, 在高负载或长时间运行条件下测试库的稳定性和性能;
- 5) 回归测试。a) 历史问题复现, 针对之前发现的 Bug, 编写测试用例并确保问题不再复现。b) 版本对比测试, 在新版本发布前, 对比旧版本的测试结果, 确保没有引入新的问题。

8.2 性能测试完整性

共性算法库的性能测试需要对每个算法编写完整的单元测试, 测试算法库性能, 具体如下:

- 1) 基准测试, 使用标准数据集或合成数据测试算法的性能, 确保其满足性能要求;
- 2) 资源使用测试, 测试算法在运行时的内存占用、CPU 使用率等资源消耗;
- 3) 并行性能测试, 如果支持多线程或分布式计算, 测试并行化后的性能提升和扩展性。

8.3 兼容性测试完整性

共性算法库的兼容性测试需要针对算法库编写完成的单元测试, 测试算法库性能, 具体如下:

- 1) 跨平台测试, 在不同操作系统 (如 Linux、Windows) 和硬件架构 (如 x86、ARM等) 上测试算法库的行为;
- 2) 编译器测试, 使用不同编译器 (如 GCC、Clang、MSVC) 编译并测试库的正确性;
- 3) 多语言绑定测试, 如果库支持多语言绑定 (如 C++、Fortran), 测试各语言接口的正确性。

9 代码规范

9.1 代码格式规范

- 1) 命名规范, 变量、函数、常量、类型等命名应清晰、一致, 遵循统一的命名规则 (如驼峰命名、下划线命名)。
- 2) 代码格式, 使用统一的代码格式化工具 (如 clang-format、astyle) 确保代码风格一致。缩进、空格、换行等应符合团队或项目约定。
- 3) 注释规范, 为关键代码、复杂逻辑、算法实现添加清晰的注释。
- 4) 错误码规范, 定义统一的错误码, 并在接口中返回。
- 5) 异常处理, 在底层库中避免抛出异常, 而是通过返回值或错误码处理异常情况。
- 6) 日志记录, 使用统一的日志库记录关键事件和错误信息, 便于调试和监控。

9.2. 性能与资源管理规范

- 1) 性能优化, 在保证正确性的前提下, 优化算法的时间复杂度和空间复杂度。
- 2) 资源管理, 确保资源 (如内存、文件句柄) 的正确分配和释放, 避免内存泄漏。
- 3) 线程安全, 如果支持多线程, 确保算法库的线程安全性。

9.3. 版本控制与协作

- 1) 版本管理：使用版本控制工具（如 Git）管理代码，遵循分支管理规范（如 Git Flow）。
- 2) 提交信息规范，提交信息应清晰描述变更内容，遵循统一的格式（如<类型>: <描述>）。
- 3) 代码审查，通过代码审查（Code Review）确保代码质量和规范性。
- 4) 发布管理，遵循语义化版本控制（Semantic Versioning），明确版本号的含义（如 MAJOR.MINOR.PATCH）。